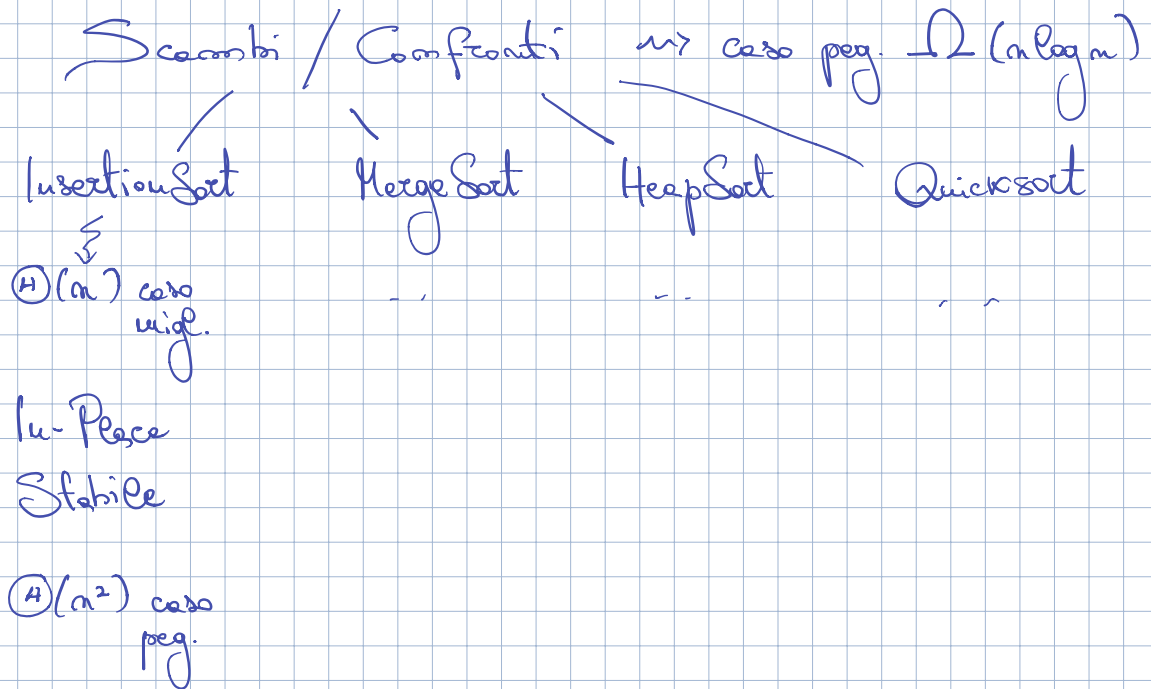


Ordinamenti



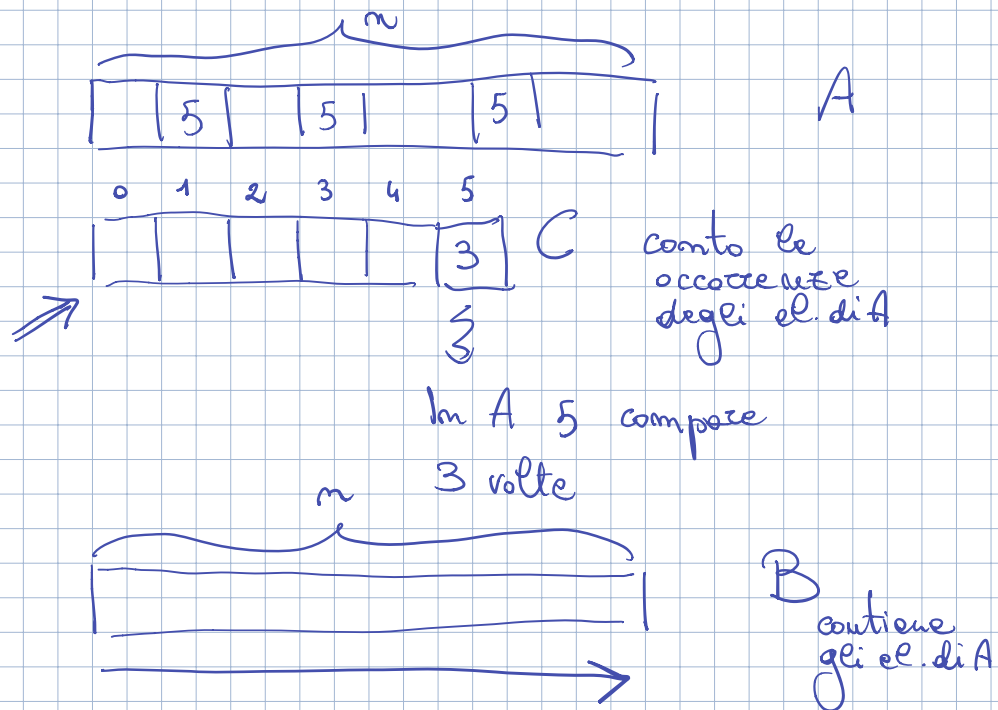
Ordinamenti in Tempo Lineare

$\Downarrow$
Richiedono ipotesi sull'input

Counting Sort
Radix Sort
Bucket Sort

Counting Sort

Idea



A vettore di interi

Se A contiene interi compresi tra 0 e K

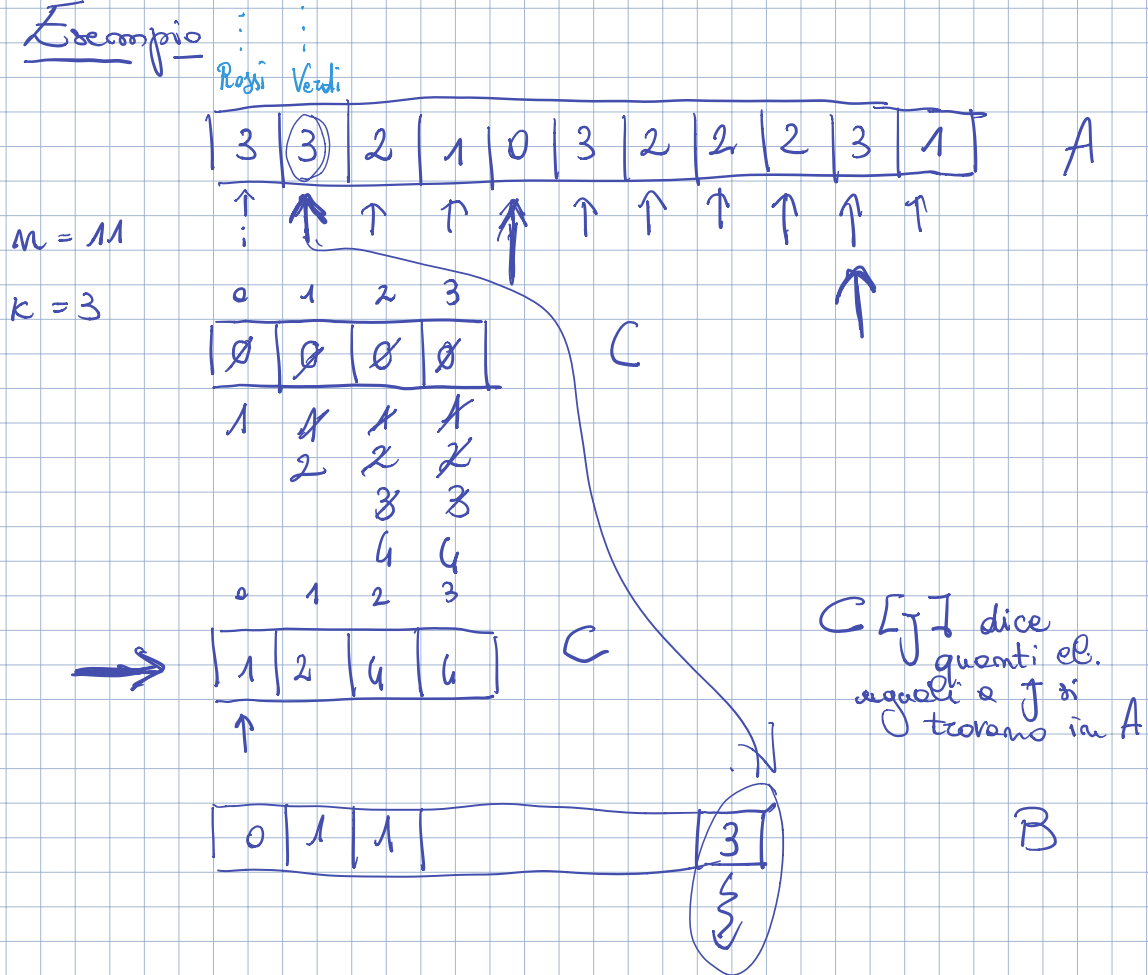
C ha lung. $K+1$

- Come utilizzo C ?
- Come ottengo B ?

Come utilizzo C

- Inizializzo C con tutti 0
- Scandisco A e incremento i valori in C

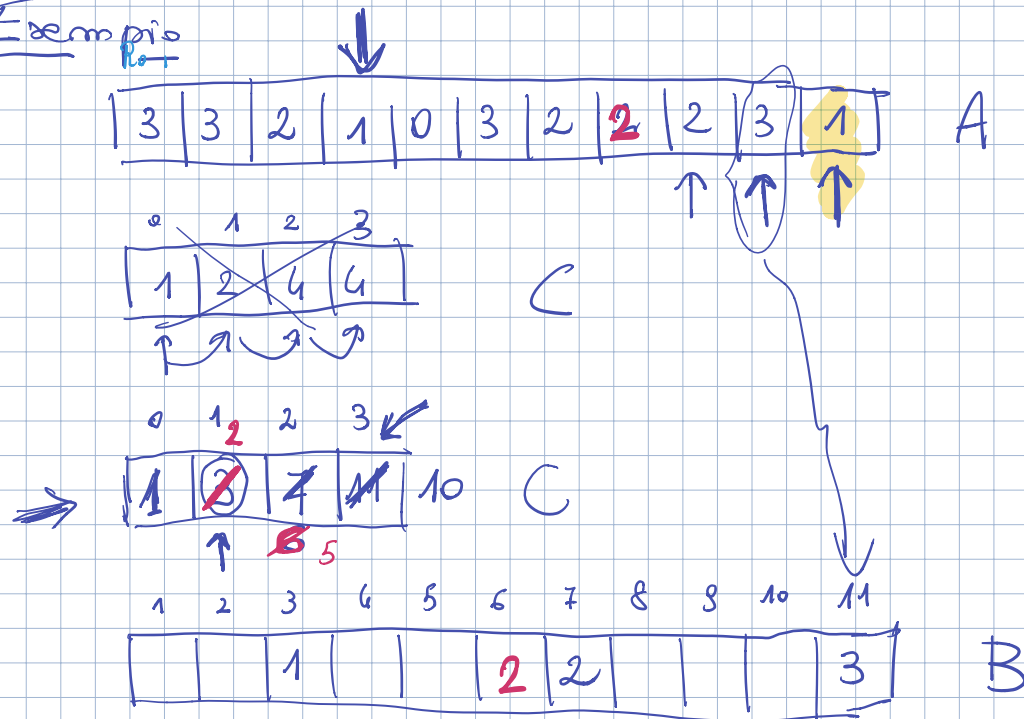
Esempio



Il vettore C deve permettermi di stabilire dove copiare in B ogni singolo elemento di A
Vorrei che l'algoritmo fosse stabile

- Modifico il vettore C in modo che
 in $C[j]$ sia scritto quanti elementi
 minori o uguali a j si trovano in A
 (Scansione di C)

Esempio



- Ottengo B scandendo A da dx verso sx
 inserendo $A[i]$ in posizione $C[A[i]]$
 nel vettore B e poi decrementando
 $C[A[i]]$

CountingSort (A, B, k) {

④(k) $C \leftarrow \text{new_array}(k+1) \quad \% C[0..k]$

④(k) for ($j \leftarrow 0$ to k) {
 $C[j] \leftarrow 0$ % perché?
}

④(n) for ($i \leftarrow 1$ to $A.\text{length}$) {
 $C[A[i]] \leftarrow C[A[i]] + 1$
}

④(k) for ($j \leftarrow 1$ to k) {
 $C[j] \leftarrow C[j] + C[j-1]$
}

④(n) for ($i \leftarrow A.\text{length}$ downto 1) {
 $B[C[A[i]]] \leftarrow A[i]$
 $C[A[i]] \leftarrow C[A[i]] - 1$
}

Stabile

}

Non In-Place

Se $k \in O(n)$

④($n+k$) = ④(n)

④($n+k$)

$\approx$
 $k \gg \text{Max}(A)$

Esempi

- K costante

Dato A di lung. n i cui elementi sono interi
nel range $[1..5]$
 $\frac{1}{3}$

- $K \leq n$

Dato A di lung. n sono tutti
nel range $[0..n]$
 $\frac{1}{2}$
 $K = n \in O(n)$ sì

- $K \leq 100 \cdot n$

$$O(n)$$

$$[0..100 \cdot n]$$

$$K \leq 10 \cdot n + 5 \sqrt{n}$$

- $K \leq 10 \sqrt{n}$

- $K \leq 3 \log n \leftarrow$

- $K \leq n^2 \leftarrow$ ~~Counting Sort costa~~

$$\Theta(n + n^2) = \Theta(n^2)$$

Usa Merge Sort

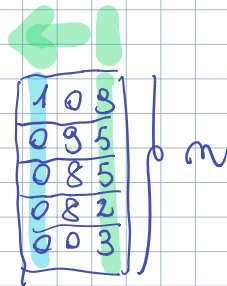
- $K \leq n \cdot \log(\log n) \rightsquigarrow$ conviene Counting Sort
Ha mem $\Theta(n)$

Radix-Sort

Ordinare un vettore A di lunghezza n contenente interi aventi d digit

Es.

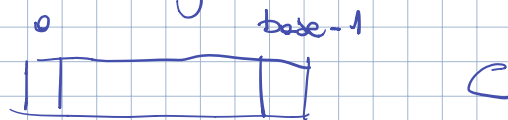
$d=3$



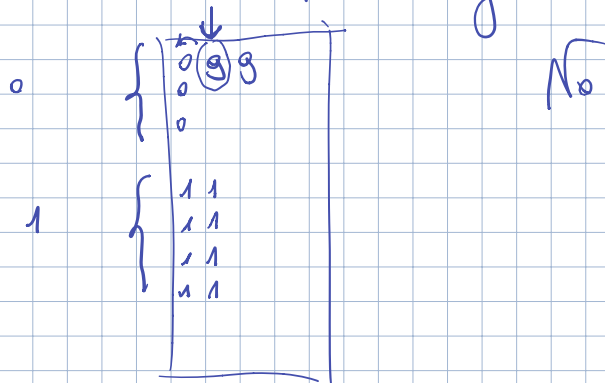
1	0	8
0	9	5
0	8	5
0	8	2
0	0	3

Idea

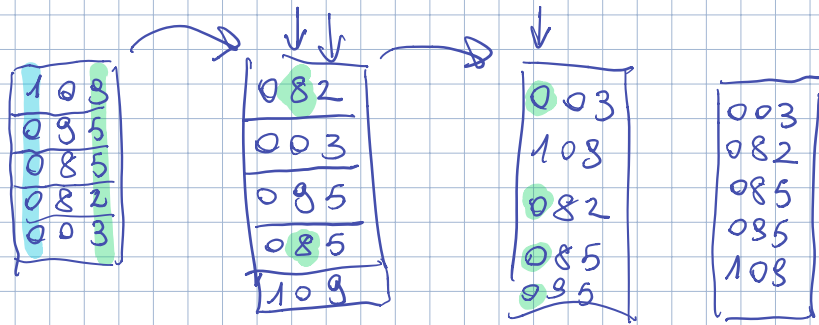
Usare CountingSort sulle singole cifre



Partendo dalle più sig.



Parte della meno significativa
Counting Sort è stabile



Radix-Sort

Ordina usando Counting Sort
 sui singoli digit a partire dal
 meno significativo

$$\begin{aligned} & \mathcal{O}(d) \cdot \mathcal{O}(n + \text{base}) \\ &= \mathcal{O}(d \cdot n) \end{aligned}$$

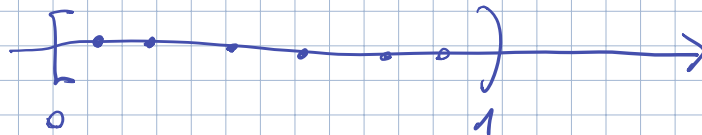
$\nwarrow$
 numero
 di digit

Se d è costante $\mathcal{O}(n)$ Lineare

Se $d \leq \log(\log n) \leadsto \mathcal{O}(\log(\log n) \cdot n)$

Bucket Sort

A vettore con elementi nel range $[0, 1)$
Uniformemente distribuiti nell'intervallo $[0, 1)$



Idea

- Divido $[0, 1)$ in n sottointervalli
- Scandisco A e colloco ogni elemento nel suo sottointervallo
- Ordino gli elementi di ogni sottointervallo
- Concateno gli ordinamenti dei sottointervalli

Esempio

